EPL484 / SOFTWARE EVOLUTION / PROJECT 1

Guava
Software evolution and architecture recovery

A reproducible study of thirty sporadically selected stable Java releases, revised with the instructor extractor, one release per major/minor family, a pilot-calibrated k rule, and the supplied paper’s Usage Equality method.

HISTORY	EXPERIMENT	BOUNDARY
30 stable releases 10.0 → 33.7.2	**120 final partitions** ACDC and KMeans × full/filtered	**Internal classes only** Merged inner/anonymous types
30 distinct major/minor families	Lecture MQ and MoJoFM agreement	Instructor extractor; project-owned code

Study boundary and evidence

The observed graph grows from **317** to **594 canonical classes**, and from **1,289** to **2,750 directed internal dependencies**. These are measurements of a static implementation view. They are not a claim about Guava's intended architecture or its users' experience.

Release window: 2011-09-28–2026-09-29. Dataset snapshot: 2026-10-17. Repository: github.com/google/guava.

All measurements in this document are loaded from saved result files. Source code, raw inputs, exact assignments and a machine-readable audit trail accompany the report. Student names and contributions must be supplied truthfully by the submitting team.

1 Revision record and controlled interpretation

This report supersedes the original study published at commit d970305. The original configuration, measurements and full repository history remain identifiable; compact tables are preserved under results/baseline_v1. Revised observations are recomputed from the original release artifacts under the following declared changes. [D17]

Concern	Revised decision
Custom annotation-heavy class graph	Use the unmodified instructor extractor as the sole primary edge source. Keep custom member extraction separate for UE.
Public-suffix ownership	Include Guava-owned com.google.thirdparty.publicsuffix implementation, alongside com.google.common.
Repeated patch families	Select 30 distinct major/minor families from 45; one stable representative per family.
Weak square-root k heuristic	Choose one middle-release inertia elbow, then freeze a size-calibrated k rule for the final series.
Lecture evaluation alignment	Add directional MoJoFM agreement; retain supplemental ARI and the lecture-supported loop-free MQ formula.
UE versus older SIG detector	Retain pure UE from the supplied paper; adding SIG would be a separately justified hybrid.

Latest release measurement	Original study	Revised study
Canonical classes	591	594
Primary directed edges	3,969	2,750
UE removals	22	22
ACDC/full clusters	95	135
KMeans/full clusters	18	43

At the same revised ownership scope, the instructor graph has 2,750 edges and the supplemental broad graph has 3,982. They share 2,750; 1,232 are supplemental-only, including 1,145 targeting annotation types. The class CSV and comparison files preserve both models without mixing their edges. [D6]

The whole-study revision changes sampling, scope, graph policy and k together. A difference between its aggregate scores and the original run cannot be attributed to a single change. The same-version extractor comparison above controls the class scope; it is distinct from the longitudinal before/after narrative. Better alignment and reproducibility do not prove that a recovered partition is the intended architecture. [D17]

2 Findings and experiment map

Structural scale increases across the sampled history. Canonical classes change by 87.4% and internal directed dependencies by 113.3% between the endpoints. Irregular sampling describes broad changes; it does not identify the exact intervening commit that caused them. [D1; L12 pp.35–48]

Usage Equality produces an explicit, inspectable exclusion set. The replacement detects 6–23 omnipresent classes per release; 31 distinct class identities are detected at least once. Filtering removes those nodes and their incident edges from the analysis graph. It does not delete code from Guava. [D1; D2]

ACDC, after holding the evaluated graph fixed: reclustering the retained graph increases MQ in 0 sampled releases, decreases it in 30, and leaves it unchanged within 10^{-12} in 0. This is a structural metric result, without an authoritative architecture against which to measure accuracy. [D3]

KMeans, after holding the evaluated graph fixed: reclustering the retained graph increases MQ in 29 sampled releases, decreases it in 1, and leaves it unchanged within 10^{-12} in 0. This is a structural metric result, without an authoritative architecture against which to measure accuracy. [D3]

Condition	Algorithm	Graph evaluated
A1	Original ACDC (+so, -l20)	All canonical internal classes
A2	Same ACDC configuration	Induced graph after UE removal
A3	Sparse normalized KMeans	All canonical internal classes
A4	Same features and version-derived k	Induced graph after UE removal

The assignment requires all four conditions. The additional note requesting one analysis per version is implemented as **one final partition per condition per release**, following a single-version pilot. KMeans' ten internal initializations produce one fitted result. The latest-release AI experiment is reported separately, as required by the assignment. [AS pp.2–4]

The recorded AI proposal groups the latest 594 classes into 18 named modules. Its full-graph MQ is 0.0943 versus 0.1790 for KMeans and 0.2553 for ACDC. The AI also sees names and package documentation, so this is not a controlled test of superiority from identical information. Its conceptual boundaries and confidence limits are discussed alongside the numerical result. [D12]

3 Subject and sporadic sampling

The students selected **google/guava**. The study follows the published main Guava JRE artifact, providing a consistent production boundary across a long release history. This is a library's implemented class architecture; other repository modules are outside the sampled artifact. [G1; D4]

Course eligibility criterion	Recorded evidence
Java; latest release ≥10,000 LOC	GitHub primary language: Java. 96,622 nonblank, noncomment lines in owned Java sources.
GitHub repository created before 2024	2014-05-29
Not a fork; not archived	fork=false; archived=false at the recorded snapshot.
≥50 commits since 2024-01-01	100 distinct qualifying commits retained as a lower-bound witness, rather than an inferred total.
≥10 major/minor releases	30 stable releases covering exactly 30 distinct major/minor families.

The eligibility SLOC count includes source-owned package descriptors. The narrower source-file population linked to canonical class nodes contains 96,546 code lines at the latest release. Both exceed the requirement; the denominators are declared rather than treated as interchangeable. [D4; D6]

Selection is specified before metric inspection

The captured tag catalog contains 124 refs, of which 82 match the numeric stable-release pattern. Forty-two nonmatching refs include prereleases, auxiliary artifacts and alternate tag names; they are not all beta releases. Nine numeric releases before 10.0 are outside the defined sampling window. The ambiguous 13.0 duplicate naming and two later backports to earlier lines are also excluded, leaving 70 chronological mainline candidates. [D4]

Family rank for sample i: **round(i × 44 / 29)**, i = 0,...,29

The 70 eligible releases form 45 major/minor families. The rule selects 30 distinct families, with one stable representative each. Use the highest eligible patch within a selected family, except that the earliest family anchors the study at 10.0; the newest ends at 33.7.2. This eliminates repeated observations from heavily patched families. [D4]

Spacing is uniform in family rank, not calendar time. Some selected families are adjacent; 30 observations cannot all skip a family when only 45 families exist. A patch representative is permitted, but a family appears only once. No release is removed because its graph is identical or its MQ is inconvenient. Plain artifacts precede 23.1 and JRE artifacts follow the variant split; Android is not a separate historical observation. [D4]

Eligibility evidence does not establish instructor approval. No teacher approval is represented as observed. The publication dates of old releases come from Guava's release-history pages; later dates use GitHub Release publication timestamps. Git commit dates remain separate fields. [AS p.1; G2; D4]

4 Data acquisition and provenance

Each selected release has an immutable Git commit identity and three Maven Central artifacts: the production binary JAR, its source JAR and the POM. Acquisition reads public data and never executes upstream Guava code. Published binaries avoid historical build failures becoming a hidden release-selection criterion. [D4; D5]

Evidence layer	Stored and checked
Release identity	Git tag name, 40-character commit SHA, published date, date basis and artifact coordinate.
Artifact integrity	Pinned SHA-256 for every downloaded file; Maven SHA-1 sidecars; ZIP CRC checks; POM version/artifact identity.
Source ownership	Per-source-file path and checksum, source-owned class inventory, SLOC definition and excluded binary entries.
Source-to-Git audit	Every Java source-JAR file at the oldest and newest selected endpoints compared byte for byte with the pinned Git archive.
Offline reproducibility	Cached artifacts must match existing pins; corrupt inputs are not silently overwritten.

Endpoint	Java files	Git/source result	Commit prefix
10.0	326	All byte-identical	baf008c5a3de
33.7.2	611	All byte-identical	666338cc49d5

All **90 artifacts** (120.41 MiB) passed pinned checks. Full Git-source correspondence was checked at the two endpoints; the intermediate 28 releases have artifact and Git identity pins but are not claimed to have the same complete per-file Git comparison. SHA sidecars fetched from the same repository provide integrity evidence, not independent authorship authentication. PGP signatures were not verified. [D5]

How to explain the data in the oral exam

Start at a row in `config/versions.json`. Show its tag SHA and Maven URL, hash the cached JAR, then inspect the source manifest and extractor provenance. Finally show how the canonical class and edge CSVs become the algorithm inputs. This chain makes an individual measurement traceable back to a concrete upstream artifact. [L4 pp.11–15]

Audit pointers: config/versions.json; data/provenance/SHA256SUMS; acquisition.json; source_inventory_<version>.json; git_source_match_10.0.json; git_source_match_33.7.2.json. All paths are repository-relative.

5 Instructor graph and ownership model

The unmodified instructor DependencyExtractor is the sole source of primary class edges. A supplemental ASM pass supplies a complete owned-class inventory, enclosing aliases and resolved field/method evidence for UE. It does not augment the instructor graph with its broader annotation, type or declaring-owner relationships. [TUT p.6; D6]

The instructor receives a deterministic JAR containing owned base class entries copied byte for byte from the original Maven binary. Per-entry hashes establish that no retained class body was rewritten. The original full artifact remains preserved. An explicit classpath launches the original CompCalc.MainClass with its unchanged bundled libraries. [D6]

Evidence	Role
classes.csv; class_aliases.csv	Source-owned common/publicsuffix types; all genuine nested/local/anonymous identities merged by enclosing metadata.
instructor_raw.csv	Original seven-category output, including occurrence counts retained for auditing.
dependencies_typed.csv; dependencies.csv	Normalized instructor relationships; one typed pair, then one directed pair; no self-loops or external endpoints.
member_dependencies.csv	Separate resolved method/field evidence for UE; original binary member identities remain distinct after class merging.
supplemental_dependencies.csv	Explicitly separate broad graph for sensitivity auditing; never added to the primary graph.

All seven tutorial relations are selected: EXTENDS, IMPLEMENTS, METHOD_CALL, METHOD_PARAMETER, FIELD_REFERENCE, TYPE_USAGE and EXCEPTION_THROWS. The primary graph accepts the constructs actually recognized by the supplied executable; it does not claim complete Java or runtime coverage. Repeated occurrences and category overlap do not multiply edge weights. [TUT p.6]

Class processing	10.0	33.7.2
Owned binary classes before merging	1,157	1,948
Nested/anonymous identities merged	840	1,354
Canonical project-owned types	317	594
Owned annotation types	6	31
Primary-graph isolates	15	45

Both com.google.common and Guava-owned com.google.thirdparty.publicsuffix sources are included. The public-suffix package carries Guava Authors headers; namespace naming alone was an insufficient exclusion rule in the original study. JDK, failureaccess and external libraries remain excluded at both edge endpoints. [D4; D6]

Owned annotation types remain nodes, including isolates. Supplemental annotation-application edges are not added. A Guava API marked @Beta is still part of a stable release. Module/package descriptors and multi-release overrides are excluded. Static analysis omits arbitrary reflection, erased constants and full virtual-dispatch expansion. [D6]

6 Replacement JNode: Usage Equality

JNode-UE implements Yano and Matsuo's Usage Equality method. It is the replacement requested for this assignment, grounded in the supplied 2020 paper. It is not a reproduction of the older JNode/Noise method based on ComponentRank and shortest-path significance. This methodological substitution is explicit. [UE pp.663–664; L67 pp.46–47; D2]

$$D_M(A,B) = \Sigma_{m \in M(A,B)}\ 1 / [xm_fanin(m) \times mx(B)]$$
$$e(B) = 1 - \max_A D_M(A,B)$$
$$c(B) = 1 - 1 / fanin(B)$$

Remove B exactly when **e(B) ≥ 0.85** and **c(B) ≥ 0.90**.

Term	Precise meaning after class-owner normalization
M(A,B)	Distinct members of B referenced by members belonging to caller class A.
xm_fanin(m)	Distinct source members outside B that reference target member m; not caller classes.
mx(B)	Number of B's distinct members referenced from outside B; unreferenced members do not enter the denominator.
fanin(B)	Number of distinct other internal project classes with resolved member use of B.

The cardinality threshold requires at least ten caller classes. A class without incoming member use has undefined scores and is retained. Scores are computed on the original graph once; all qualifying nodes are removed simultaneously. No repeated cascade or per-release threshold search is performed. “Outside B” means another owned class, not an external library. Every score, strongest caller and decision is saved. [UE; D2]

A small example explains the difference from degree

Suppose one utility operation is referenced by one member in each of ten caller classes. Each caller's dedication is 1/10, so e = c = 0.90 and the utility qualifies. Now imagine a logger shared by 21 callers, where a startup class alone also uses a separate initialization operation. Startup's dedication is 1/(21×2) + 1/(1×2), making e approximately 0.476. That exclusive relationship prevents removal despite high fan-in. This is a constructed explanatory example, not a Guava measurement. [UE Fig.1; D2]

Implementation: set/dictionary aggregation over distinct member references; cached declaring owners; stable summation and exact rational fallback at threshold boundaries. Expected core aggregation is O(R + C + N), plus deterministic output sorting, where R is unique member references, C caller–target class pairs and N classes. No all-pairs shortest paths are required. [D2]

7 Two recovery methods, frozen rules

ACDC: the original implementation

The archived York Java implementation is compiled and invoked using `+so -l20`. SubGraph Dominator runs before Orphan Adoption. The BodyHeader pattern matches C source/header pairs and provides no Java class-level evidence here. The default fine, flat DownInducer output is preserved. [TUT pp.7–8; L67 pp.19–24; ACDC; D7]

The maximum-size parameter governs SubGraph construction, **not every final cluster**. ACDC can place remaining orphans into a large final container. This output is reported without post-hoc splitting. Dependency-only RSF cannot encode isolates, so known genuine isolates are restored as explicit singleton clusters. All other classes must appear exactly once in ACDC's output. Invalid, duplicate, missing or stale output is rejected. [D7]

KMeans: fixed structural coordinates

$$U = \text{sorted union of class identities across all 30 releases}$$
$$X = \text{row-L2-normalize}([A + I, (A + I)^T])$$
$$k_{full} = \max(1, \text{ceil}(n_{full} \times k_{pilot} / n_{pilot}))$$
$$k_{filtered} = \min(k_{full}, n_{filtered})$$

Each current class is a row in a sparse CSR matrix with **1,430 fixed columns** (two directions × 715 union identities). Absent or removed coordinates contain zero evidence. Identity entries encode self-similarity only in features; they never enter the dependency graph or metric calculation. Package names and AI cluster labels are not features. [D7]

The single middle-release inertia pilot selects k=40 for n=554 classes, implying an approximate target of 13.85 classes per cluster. Scaling that value with full-graph size is an explicit extrapolation assumption. It is not a lecture theorem, a guaranteed optimum, or a choice made by maximizing final MQ. [D7; D8]

A3/A4 share the full-release-derived k, subject to the retained-node limit. Empty graphs use k=0. A reviewer can choose --k for a separate exploratory run; the 120 final partitions continue to use one frozen policy. The matched-ACDC-k pilot diagnostic below examines granularity without using its quality result to select k. [D8]

KMeans uses k-means++ initialization, Lloyd updates, random_state=484, n_init=10, max_iter=300 and tol=10^{-4}; numerical thread pools are limited to one. Class order and output labels are canonical. Actual k, convergence warnings, iterations, inertia and library versions are recorded, so identical rows producing fewer clusters cannot go unnoticed. [SK; D7]

8 Pilot and final parameter decision

The pilot uses **Guava 25.1**, the middle selected release by sample index, chosen before inspecting its quality. ACDC uses subgraph sizes 10, 20 and 40. KMeans uses one declared candidate grid on this release only; final versions are not tuned independently. The pilot outputs are separate from the 120 final A1–A4 architectures. [D8]

Pilot configuration	k	Largest	Singles	Cohesion	Coupling	MQ
ACDC -l10	110	49	23	0.323	0.003	0.319
ACDC -l20	110	49	23	0.323	0.003	0.319
ACDC -l40	110	49	23	0.323	0.003	0.319
KMeans (pilot elbow)	40	58	0	0.165	0.005	0.160
KMeans (matched k diagnostic)	110	23	9	0.247	0.004	0.242

All three ACDC settings have identical required metrics on this pilot; retaining the documented middle setting has no measured disadvantage here. This is not a theorem of parameter insensitivity. [D8]

At matched pilot k=110, ACDC MQ is 0.3193 and KMeans MQ is 0.2424. The elbow-selected KMeans MQ is 0.1603. This diagnostic controls cluster count on this graph, not cluster-size distribution or semantic validity, and does not determine the final k. [D8]

ACDC identifier compatibility

The file-oriented upstream implementation strips a final dotted suffix to form a basename. The adapter reversibly maps p.A to p.A.java so distinct Java names cannot collapse to the same package basename. It preserves each edge, saves node_id_map.csv and decodes every membership before evaluation. The original ACDC source is unchanged. Real isolates are restored as singletons; -l20 is a SubGraph-stage limit, not a final global size cap. [D7]

Frozen configuration SHA-256:

`c2d715ba45c0ee9445e8d07bfba238c3a38d1dcf656aab909f828a7617ccb35f`

9 The single-pilot inertia curve

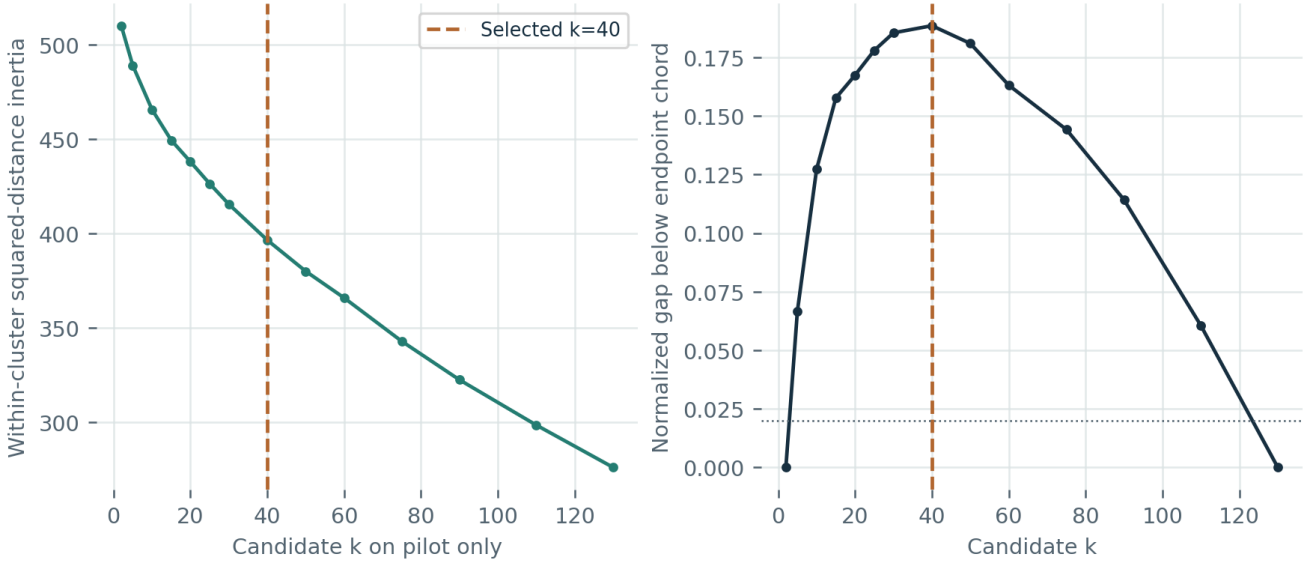


Figure 10. The fixed candidate grid's inertia and normalized elbow gap. The selected k uses inertia geometry, not MQ. Source: results/pilot/k_inertia_curve.csv.

Candidate k	Inertia	Normalized gap	Decision
2	510.1248	0.0000	
5	489.0427	0.0667	
10	465.7223	0.1273	
15	449.4790	0.1576	
20	438.0590	0.1674	
25	426.4464	0.1780	
30	415.5541	0.1855	
40	396.5607	0.1885	Selected
50	380.0554	0.1809	
60	365.9473	0.1631	
75	342.9561	0.1442	
90	322.5566	0.1142	
110	298.5641	0.0605	
130	276.1594	0.0000	

Normalize x=(k−k_min)/(k_max−k_min) and y=(I−I_last)/(I_first−I_last); gap=1−x−y. Choose the largest interior gap if it is at least 0.02; exact ties choose smaller k. Otherwise use the declared nearest candidate to k=30. Observed decision: maximum_normalized_chord_gap; selected k=40; maximum gap=0.1885. [D8]

An elbow depends on the candidate range, feature geometry and optimization fits. It estimates diminishing distance-fit returns on one graph, not the number of true components. The observed elbow is broad: neighboring k candidates have similar gaps. The subsequent size scaling preserves a target average granularity without another per-release sweep. This assumption is disclosed and actual output sizes are reported. [D8]

10 Exact lecture quality measures

The report uses the modularization-quality definition shown in the reconstruction lecture: mean cluster cohesion minus mean coupling over unordered cluster pairs. Edges are directed, binary and without self-loops. Let n_i be cluster size, μ_i its internal edge count and ε_{ij} the inter-cluster edge count in both directions. [L67 pp.15–17]

$$\text{cohesion}_i = \mu_i / [n_i(n_i - 1)] \text{ for } n_i > 1; \text{ otherwise } 0$$
$$\text{coupling}_{ij} = \varepsilon_{ij} / (2n_i n_j)$$
$$MQ = (1/k)\ \Sigma_i\ \text{cohesion}_i - (1 / \text{choose}(k,2))\ \Sigma_{i<j}\ \text{coupling}_{ij}$$

The lecture presents alternative internal-density denominators; this study follows the n(n−1) form used in its worked directed-graph example. A singleton's cohesion is zero. For k=1, coupling is zero. Empty partitions report zero by explicit convention. Every zero-edge cluster pair remains in the coupling average. These policies are declared because otherwise software implementations can compute different values from the same partition. [L67 p.17; D9]

A hand-calculable oracle

Take clusters C1={a,b} and C2={c,d}, with edges a→b, b→a, c→d, d→c and b→c. Each cluster has cohesion 2/(2×1)=1. Pair coupling is 1/(2×2×2)=1/8. Therefore **MQ=1−1/8=0.875**. Adding a separate isolated singleton gives mean cohesion 2/3 and mean coupling (1/8)/3=1/24, hence MQ=5/8. The changed score comes from the declared averaging rule, although the original five edges are unchanged. [D9]

What the numbers do and do not establish

MQ is bounded between −1 and 1 for this model. It measures structural concentration, not functional correctness, runtime performance, developer productivity or user-perceived quality. Macro averaging gives a tiny cluster the same weight as a large one; changing k, isolate handling or cluster sizes can change the score. Node-size-weighted cohesion, edge fractions and cluster-size distributions are therefore reported as diagnostics. [L67 p.16; L12 p.45; D9]

A1 and A3 share the full graph; A2 and A4 share the retained graph. Direct full-versus-filtered MQ differences change both the grouping and the population. The same-graph comparison later projects the full partition onto exactly the retained nodes and edges before evaluating it. MoJoFM and supplementary Adjusted Rand values compare partitions without treating either as ground truth. [D3; D9; D16]

11 Classes and dependencies over time

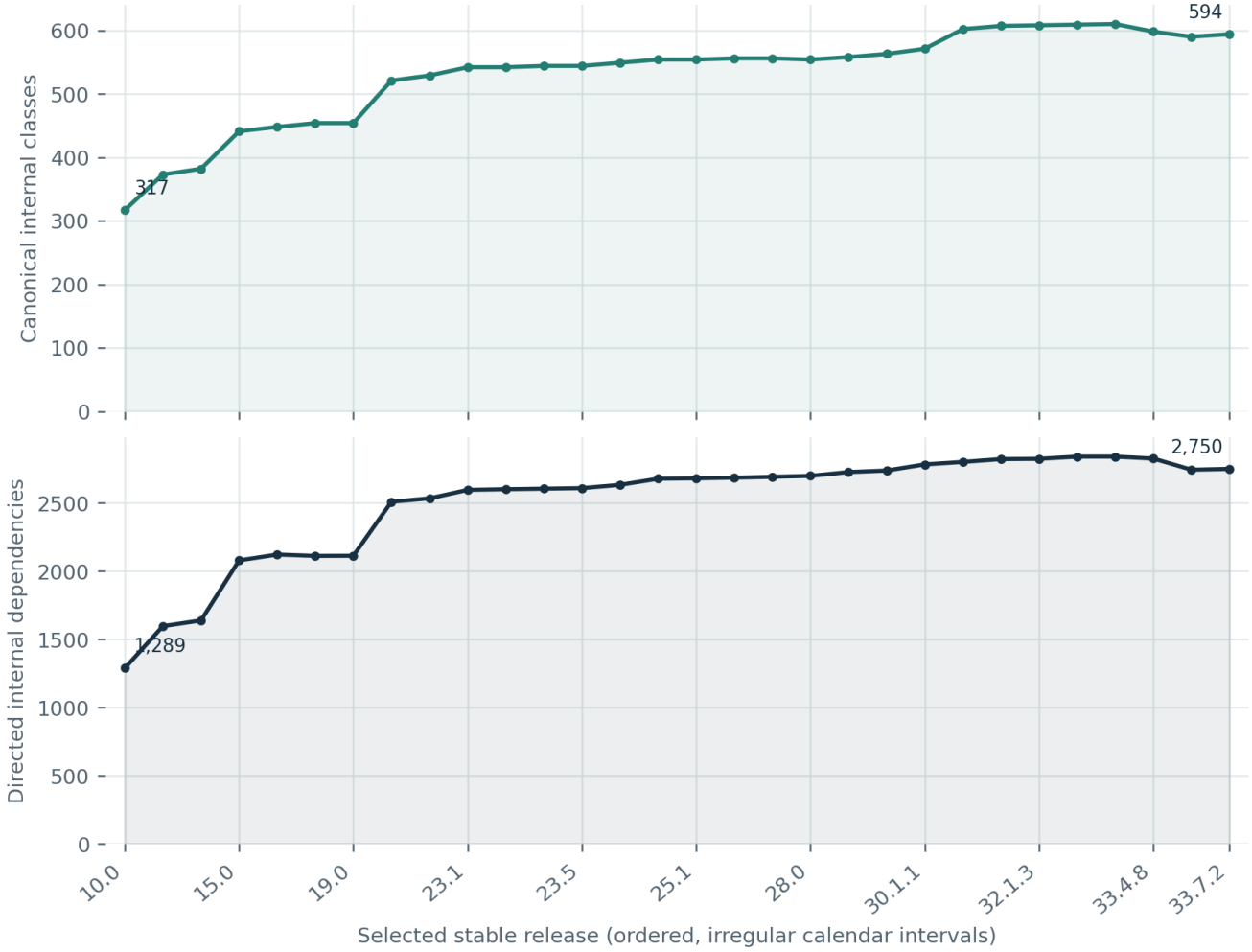


Figure 1. Canonical project-owned types and binary directed instructor dependencies for 30 distinct major/minor families. Source: results/sizes.csv.

The first selected release contains 317 classes and 1,289 dependencies; the last contains 594 classes and 2,750 dependencies. Class counts include owned annotation types and isolates. The primary edge series no longer includes supplemental annotation-application edges. [D1; D6]

The equal horizontal spacing represents ordered observations, not equal elapsed time. The family-first sample deliberately reduces repeated patch observations but is not calendar-uniform. Different compiler output and static-analysis coverage can affect observed relations. A count increase alone does not establish feature growth or worsening design. [L12 pp.43–48; D4]

Observed transition 13.0.1 → 15.0: 1,639 → 2,079 primary edges. The change-point audit preserves the added/removed identities and edges for source inspection; sparse sampling does not identify the single causing commit. [D15]

Observed transition 19.0 → 21.0: 2,113 → 2,509 primary edges. The change-point audit preserves the added/removed identities and edges for source inspection; sparse sampling does not identify the single causing commit. [D15]

12 Metadata types and graph isolates

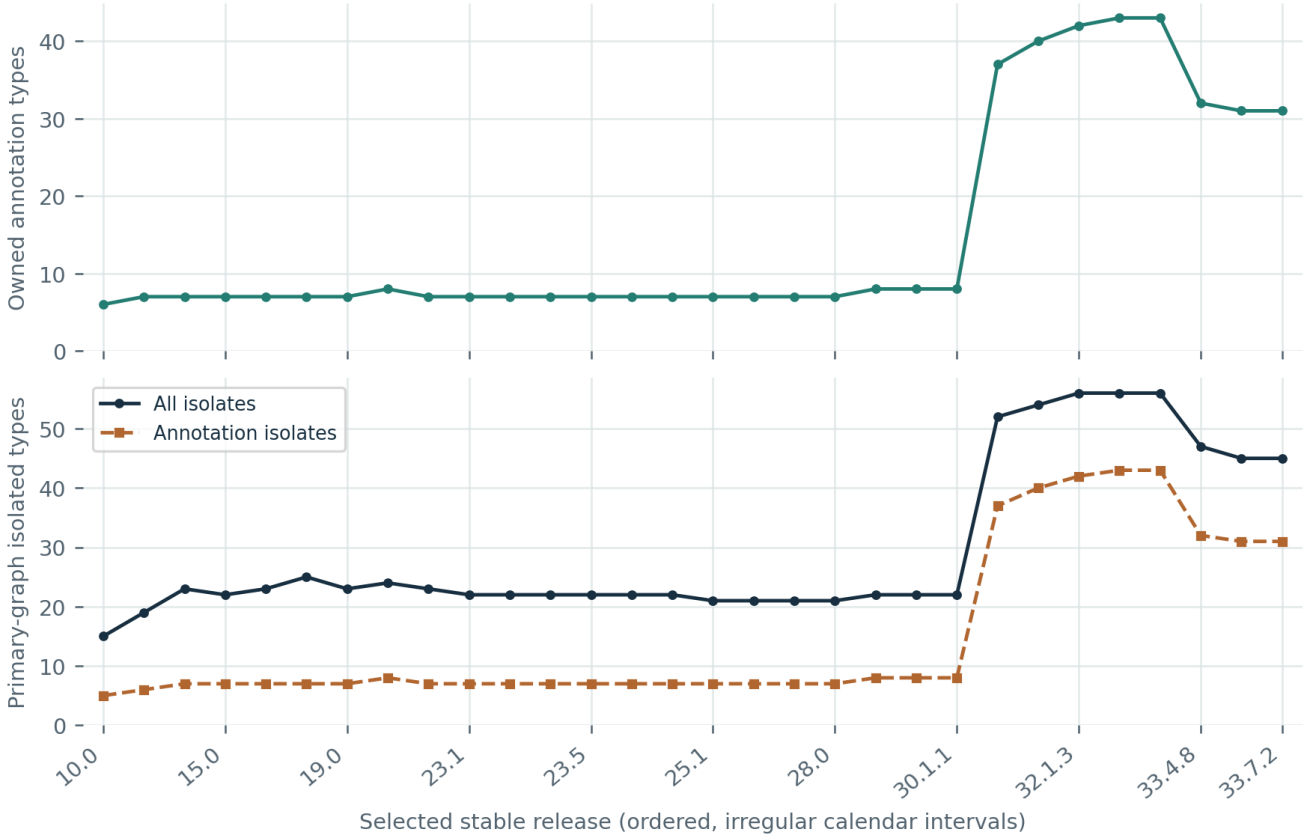


Figure 9. Owned annotation-type counts and graph isolates under the instructor baseline. Annotation types remain in the node inventory; annotation-application edges from the supplemental reader are not added. Source: classes.csv and dependencies.csv for each release.

Inventory category	10.0	33.7.2
All canonical project-owned types	317	594
Nonannotation types	311	563
Annotation types	6	31
All primary-graph isolates	15	45
Annotation isolates	5	31

Keeping all owned nodes prevents an edge-only inventory from silently discarding types. It also has a measurable consequence: ACDC restores isolates as singletons, whose cohesion is zero under the declared metric convention. KMeans receives identity-based feature rows for isolates. Metadata types can therefore still affect node, cluster and MQ counts even though their annotation applications are absent from the primary edge graph. [D6; D9]

These categories overlap: an annotation isolate is both an annotation type and an isolate; the table rows must not be added together. An absence of observed member use does not make a type a UE removal. The detector retains the no-evidence case explicitly. [D2]

13 Complete thirty-release inventory

Release	Published	Classes	Edges	UE out	Edges left	Skipped
10.0	2011-09-28	317	1,289	6	1,036	—
12.0.1	2012-07-10	373	1,597	8	1,257	5
13.0.1	2012-08-28	382	1,639	8	1,292	1
15.0	2013-09-06	441	2,079	11	1,507	2
16.0.1	2014-02-03	448	2,122	10	1,545	1
18.0	2014-08-25	454	2,112	10	1,535	1
19.0	2015-12-09	454	2,113	13	1,461	0
21.0	2017-01-12	521	2,509	16	1,692	1
22.0	2017-05-22	529	2,534	16	1,712	0
23.1	2017-09-27	542	2,596	17	1,726	1
23.2	2017-10-11	542	2,601	17	1,730	0
23.4	2017-11-09	544	2,605	16	1,759	1
23.5	2017-11-22	544	2,609	16	1,763	0
24.0	2018-02-01	549	2,633	17	1,771	1
24.1	2018-03-15	554	2,678	17	1,806	0
25.1	2018-05-31	554	2,681	17	1,809	3
26.0	2018-08-01	556	2,686	17	1,812	0
27.1	2019-03-08	556	2,692	18	1,784	2
28.0	2019-06-12	554	2,699	19	1,786	0
28.2	2019-12-27	558	2,727	19	1,810	1
29.0	2020-04-13	563	2,738	20	1,796	0
30.1.1	2021-03-19	571	2,783	20	1,881	2
31.0.1	2021-09-27	602	2,801	21	1,888	1
32.0.1	2023-06-09	607	2,822	22	1,839	2
32.1.3	2023-10-10	608	2,824	22	1,840	3
33.1.0	2024-03-13	609	2,840	23	1,840	1
33.2.1	2024-05-31	610	2,840	23	1,840	1
33.4.8	2025-04-14	598	2,826	23	1,763	10
33.5.0	2025-09-17	590	2,744	21	1,732	0
33.7.2	2026-09-29	594	2,750	22	1,724	3

Classes are canonical owned nodes; edges are distinct directed dependencies. “UE out” counts removed nodes. “Skipped” counts numeric stable tags strictly between the current and previous sampled release, including any explicitly excluded mainline/backport point in that interval. Exact intervening names, Git SHAs and artifact variants are in config/versions.json. [D1; D4]

14 What Usage Equality removes

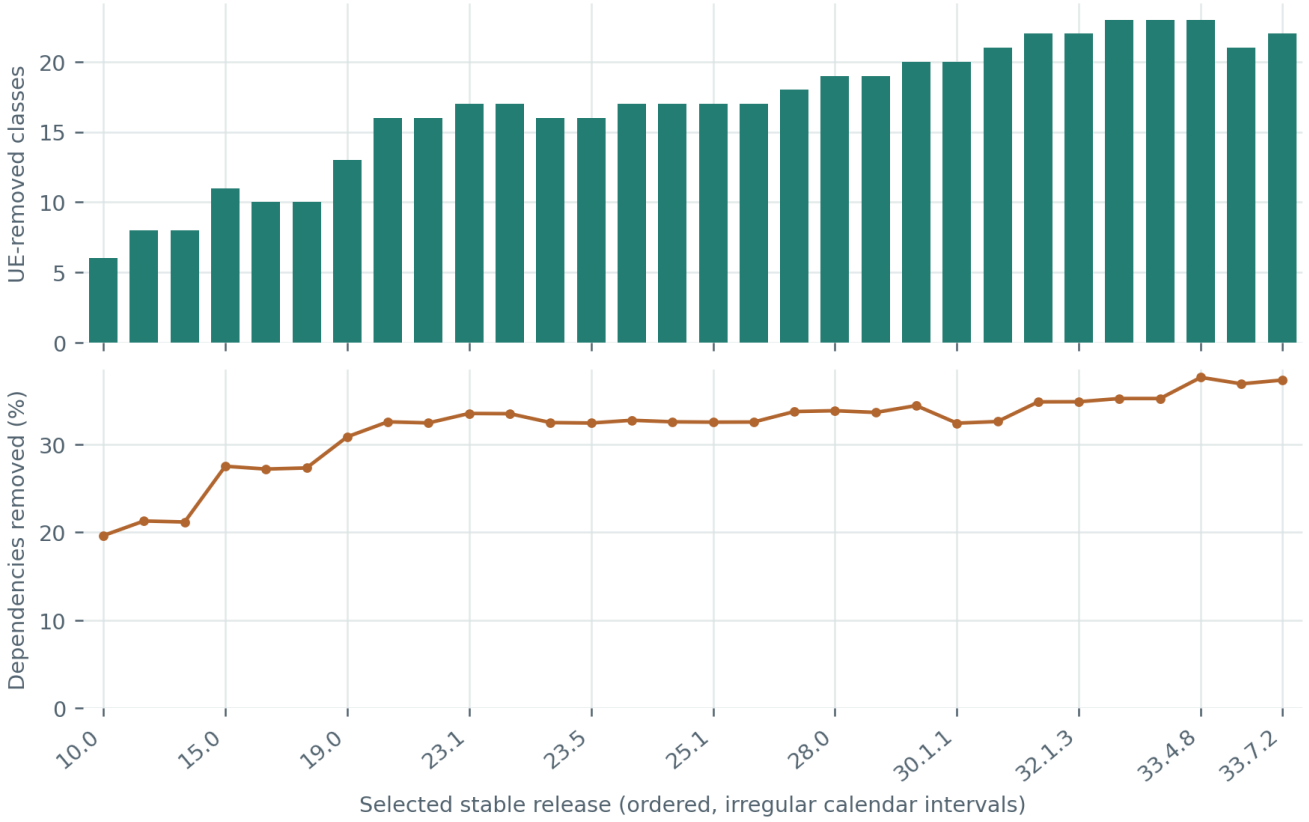


Figure 2. Detected omnipresent nodes and the fraction of original graph edges incident to removed nodes. Removal is simultaneous and uses fixed thresholds. Source: results/sizes.csv.

Frequently removed class identity	Releases detected / 30
base.Function	30
base.Preconditions	30
collect.Serialization	30
collect.UnmodifiableIterator	30

At the latest release, UE removes 22 of 594 classes and leaves 1,724 dependencies (62.7% of the full graph). A frequent removal is not proof that a class is badly designed. A utility may be useful and correct while obscuring subsystem structure for a clustering algorithm. [UE; D1; D2]

Frequency counts use all 30 releases as the denominator, including releases in which an identity does not yet exist. Full per-class scores distinguish absent classes, retained candidates and detected classes. See results/versions/<version>/ue.json and omnipresent.csv.

Latest audited target	Callers	Members	e	c	Decision
base.Preconditions	312	29	0.9276	0.9968	Remove
collect.Iterators	50	41	0.4980	0.9800	Retain

Preconditions passes both thresholds. Iterators has many callers but a strong specialized relationship with Iterables, giving a maximum dedication of about 0.502 and preventing removal. The independent case audit preserves exact fractions, all caller scores and member-CSV row evidence; names alone do not determine these decisions. [D14]

15 Modularization quality results

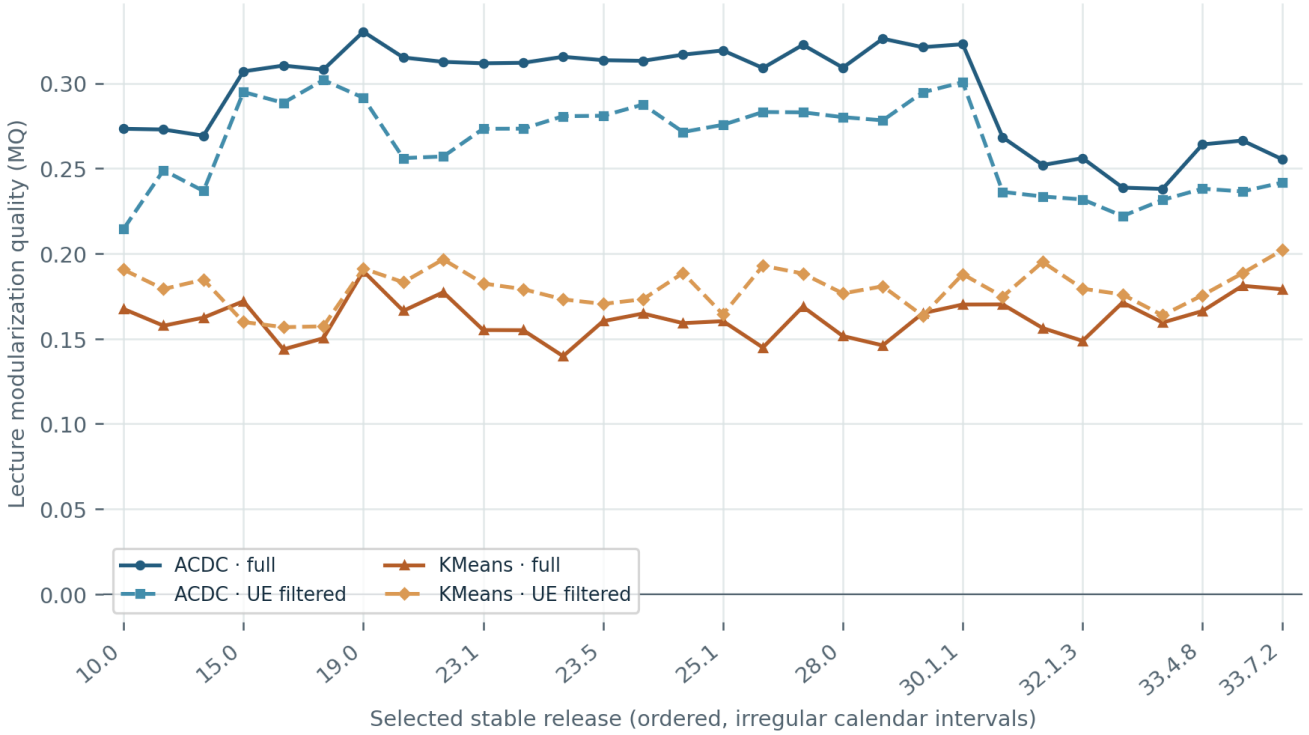


Figure 3. The four required MQ series. A1/A3 evaluate full graphs; A2/A4 evaluate retained graphs. Cross-graph gaps are descriptive and must not be read as a causal filtering benefit.

Condition	Minimum	Median	Maximum	Latest
ACDC · full	0.238	0.310	0.330	0.255
ACDC · UE filtered	0.214	0.273	0.302	0.242
KMeans · full	0.140	0.161	0.190	0.179
KMeans · UE filtered	0.157	0.179	0.202	0.202

On the same full graph, ACDC has higher MQ than KMeans in 30 of 30 releases; on the same filtered graph it has higher MQ in 30. This comparison describes the chosen metric and granularity rules. It does not demonstrate that the higher-scoring partition matches the intended subsystem architecture. Cluster counts and sizes are examined alongside the score. [D1; D9]

16 Cohesion and coupling separately

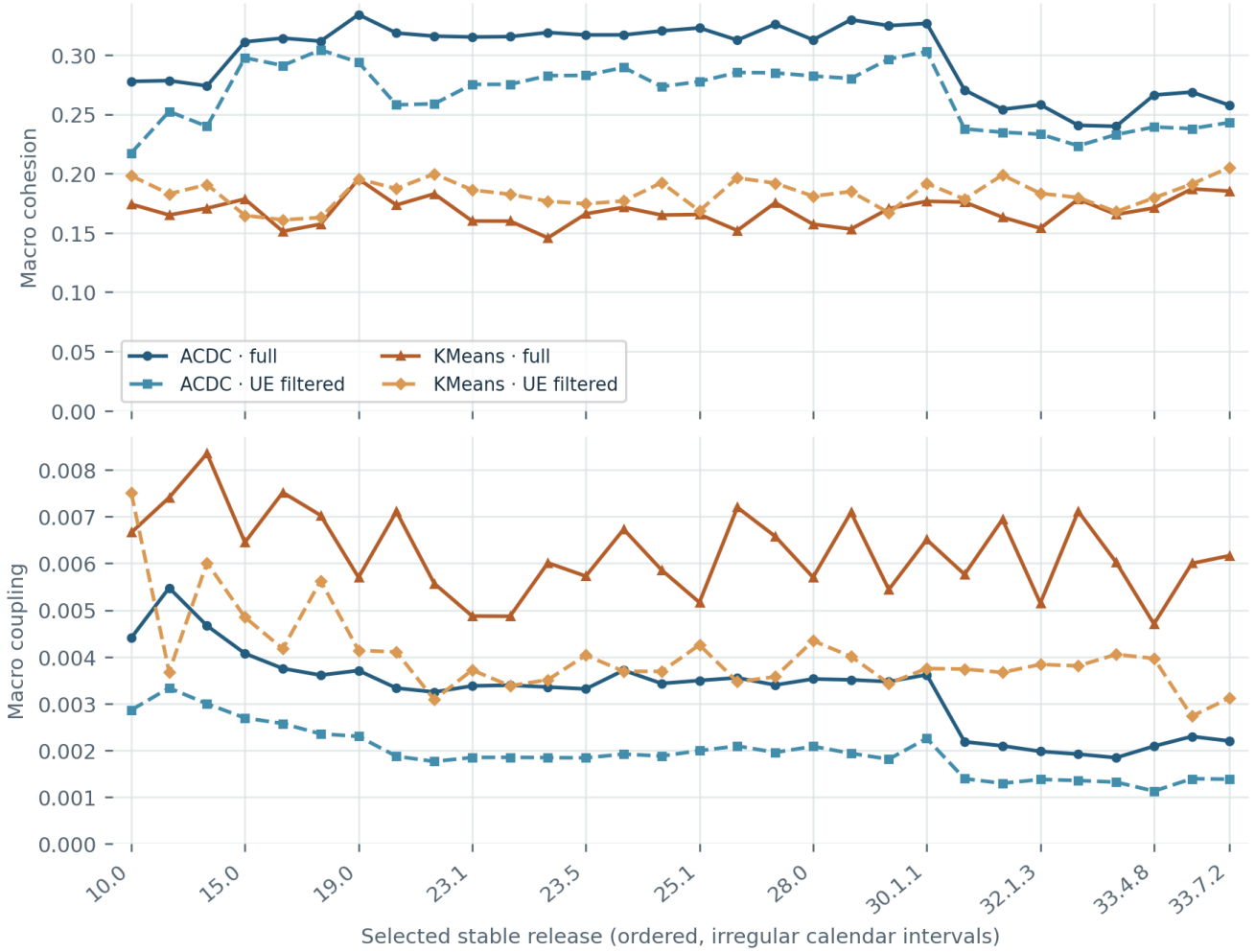


Figure 4. Required macro cohesion and macro coupling. Each panel has its own vertical scale. Both quantities average over clusters or cluster pairs, not over classes.

Showing the terms separately prevents a single MQ value from hiding its composition. A change can come from denser internal groups, fewer cross-group edges, or a change in how cluster and pair sizes are weighted. The coupling denominator also grows with the number of cluster pairs. A low macro coupling score is therefore not equivalent to having few inter-cluster edges. [D9]

For an oral explanation, choose any appendix row and open its corresponding A<n>_quality/per_cluster.csv and per_pair.csv. Sum the saved normalized terms, divide by the numbers of clusters and unordered pairs, and subtract. The report's rounded values are display values; the CSV and JSON outputs preserve the computation precision.

17 Does filtering change the grouping?

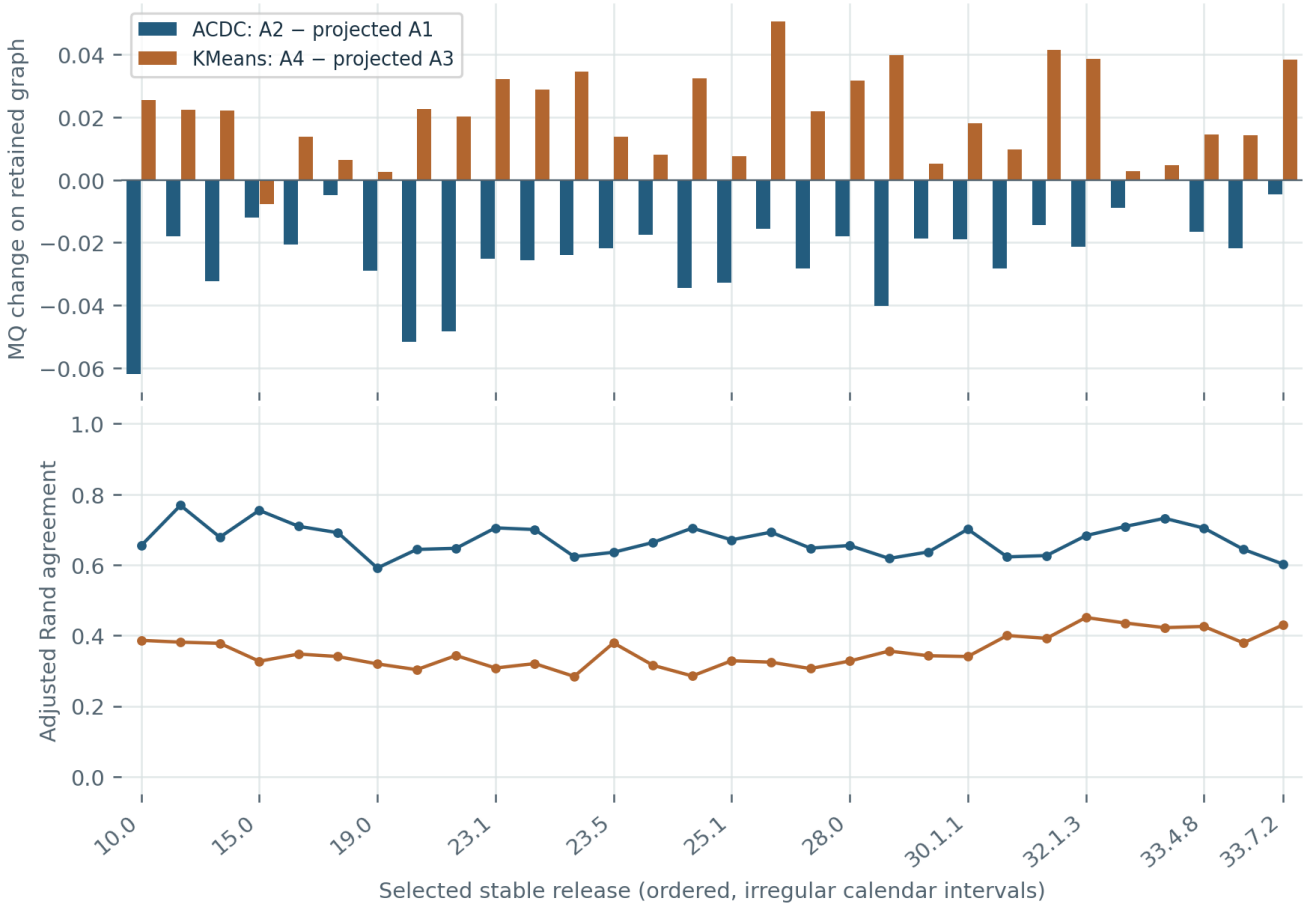


Figure 5. Recovered filtered partition versus the full partition projected to the identical retained graph. Positive MQ difference favors reclustering under the lecture metric. ARI is agreement, not correctness. Source: results/same_graph_comparisons.csv.

Algorithm	Median ΔMQ	Latest ΔMQ	Latest ARI	Δ > 0	Δ < 0
ACDC	-0.0216	-0.0047	0.602	0	30
KMeans	0.0211	0.0383	0.430	29	1

Projection removes the same classes from the full partition without reclustering the survivors. Empty projected clusters disappear. Both projected and newly recovered assignments are then evaluated using the exact same retained edges. This controls the evaluated population, making the difference attributable to the grouping produced under the declared recovery process. It still does not supply an expert architecture or prove a general benefit of removal. [D3]

18 Lecture agreement: directional MoJoFM

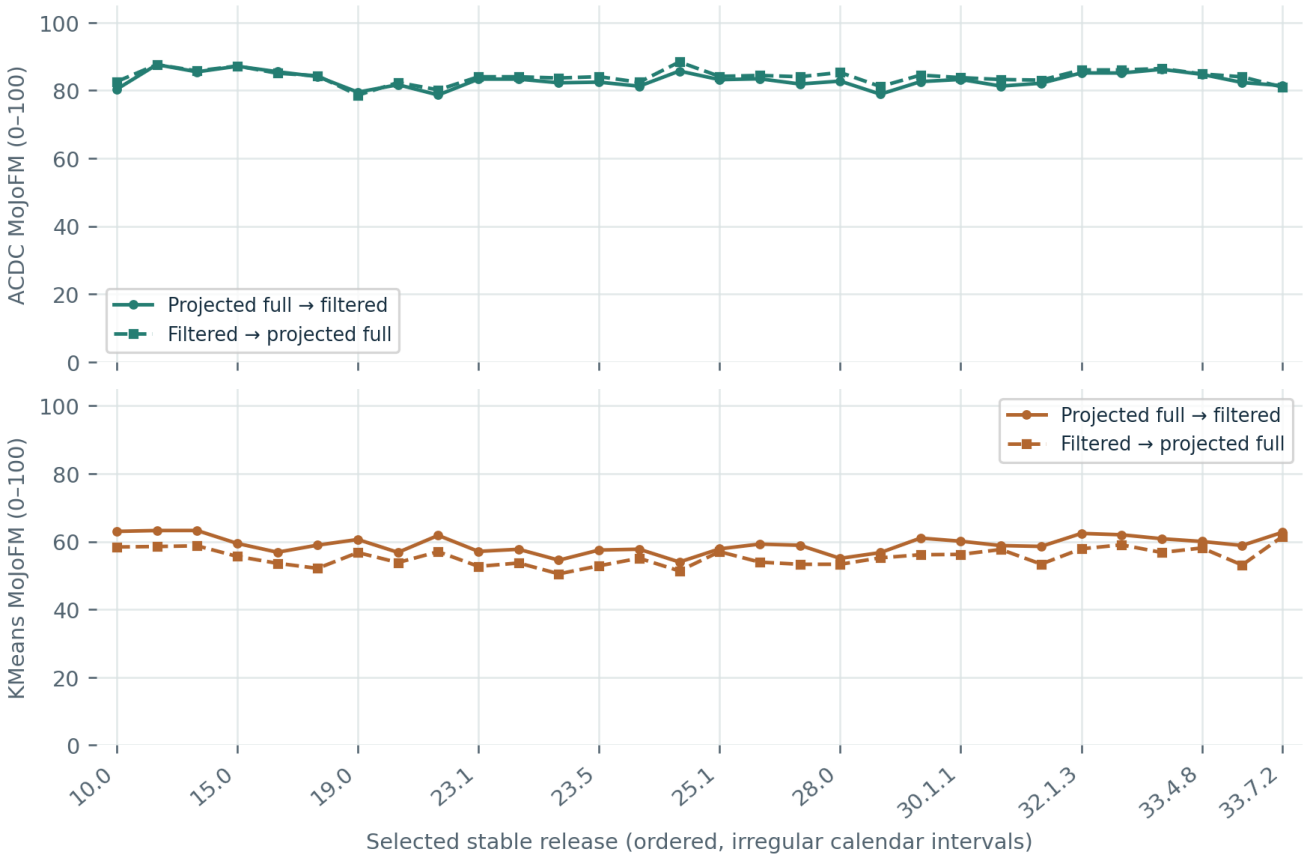


Figure 11. MoJoFM after projecting full and filtered architectures onto the same retained class universe. Forward and reverse scores use different reference partitions. Scores are agreement on a 0–100 scale, not recovery accuracy. [D16]

Latest comparison	Nodes	Forward	Reverse	ARI
A1 → A2	572	81.41	80.98	0.6025
A3 → A4	572	62.73	61.45	0.4297
A1 → A3	594	43.71	41.11	0.1876
A2 → A4	572	45.09	43.68	0.1416

$$\mathrm{MoJoFM}(P \to Q) = 100 \times [1 - \mathrm{MoJo}(P,Q) / \mathrm{maxMoJo}(Q)]$$

MoJo counts minimal moves and cluster joins toward a specified reference. Its normalizer depends on that reference, so reversing the inputs can change the result. The saved table contains distances, denominators, source/reference cluster counts, both directions and a hash of the exact object universe. Empty or one-object normalized boundaries are recorded as undefined rather than silently inventing a fraction. [L67; D16]

An expert reference is necessary to interpret a score as architecture recovery accuracy. It is not necessary to calculate agreement between two stated partitions. This corrects the original overly restrictive rationale for omitting MoJoFM. ARI remains a supplementary symmetric agreement measure; neither measure assesses module names or intended responsibilities. Temporal and AI comparisons apply the same object-universe rule. [D16]

19 Granularity and metric sensitivity

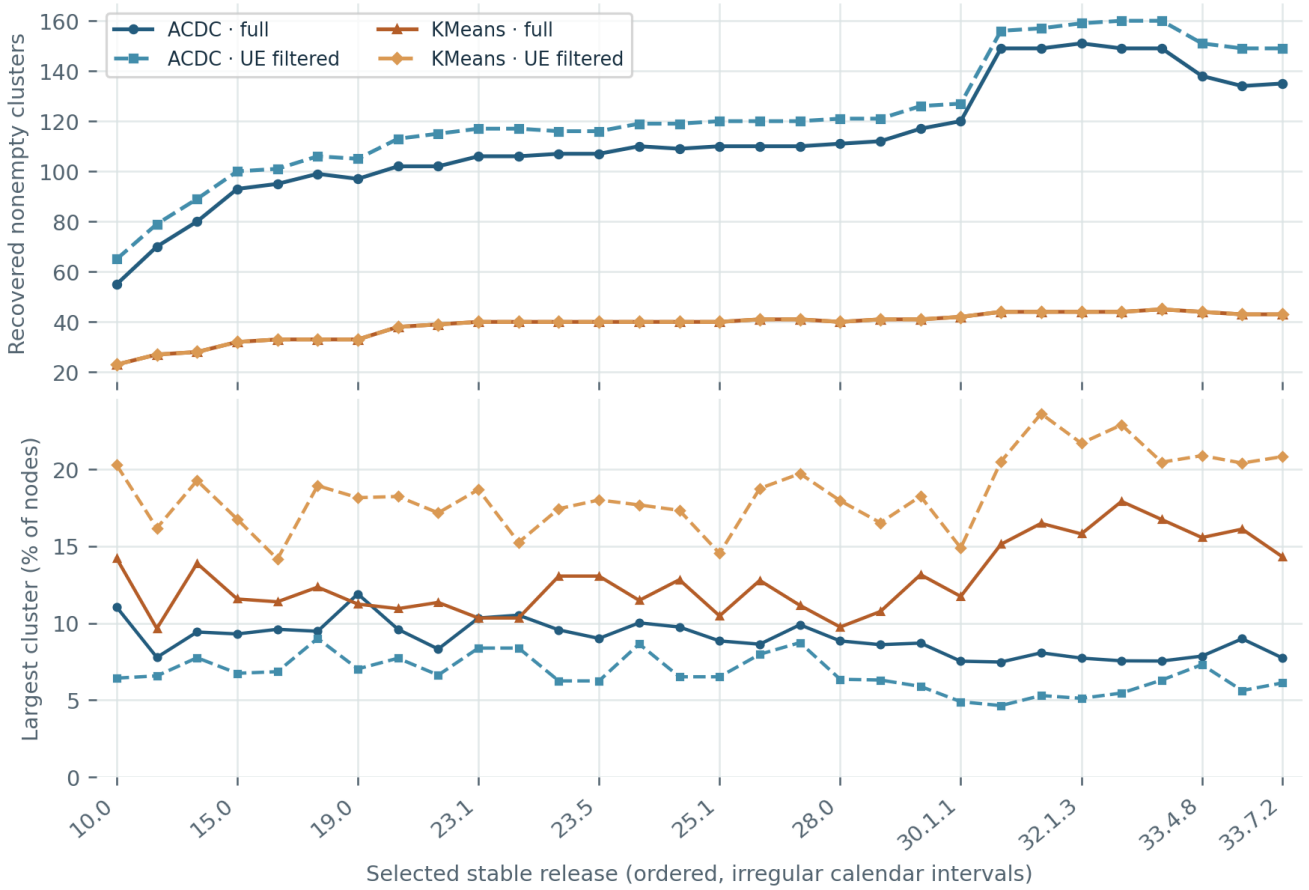


Figure 6. Number of recovered clusters and share of nodes in the largest cluster. ACDC's subgraph-size setting is not a final cluster-size cap.

Latest	k	Largest	Singletons	Size-weighted cohesion	Inter-edge share
A1	135	46	47	0.232	63.9%
A2	149	35	60	0.231	53.1%
A3	43	85	0	0.129	65.9%
A4	43	119	0	0.135	53.1%

The supplementary cohesion weights each cluster by its node count, while the required MQ weights each nonempty cluster equally. Inter-edge share counts all edges crossing the partition boundary and divides by the total edge count. Neither diagnostic is substituted for the lecture metric. Their purpose is to reveal large residual containers or numerous tiny clusters that the macro averages can conceal. [D9]

20 Structural change between samples

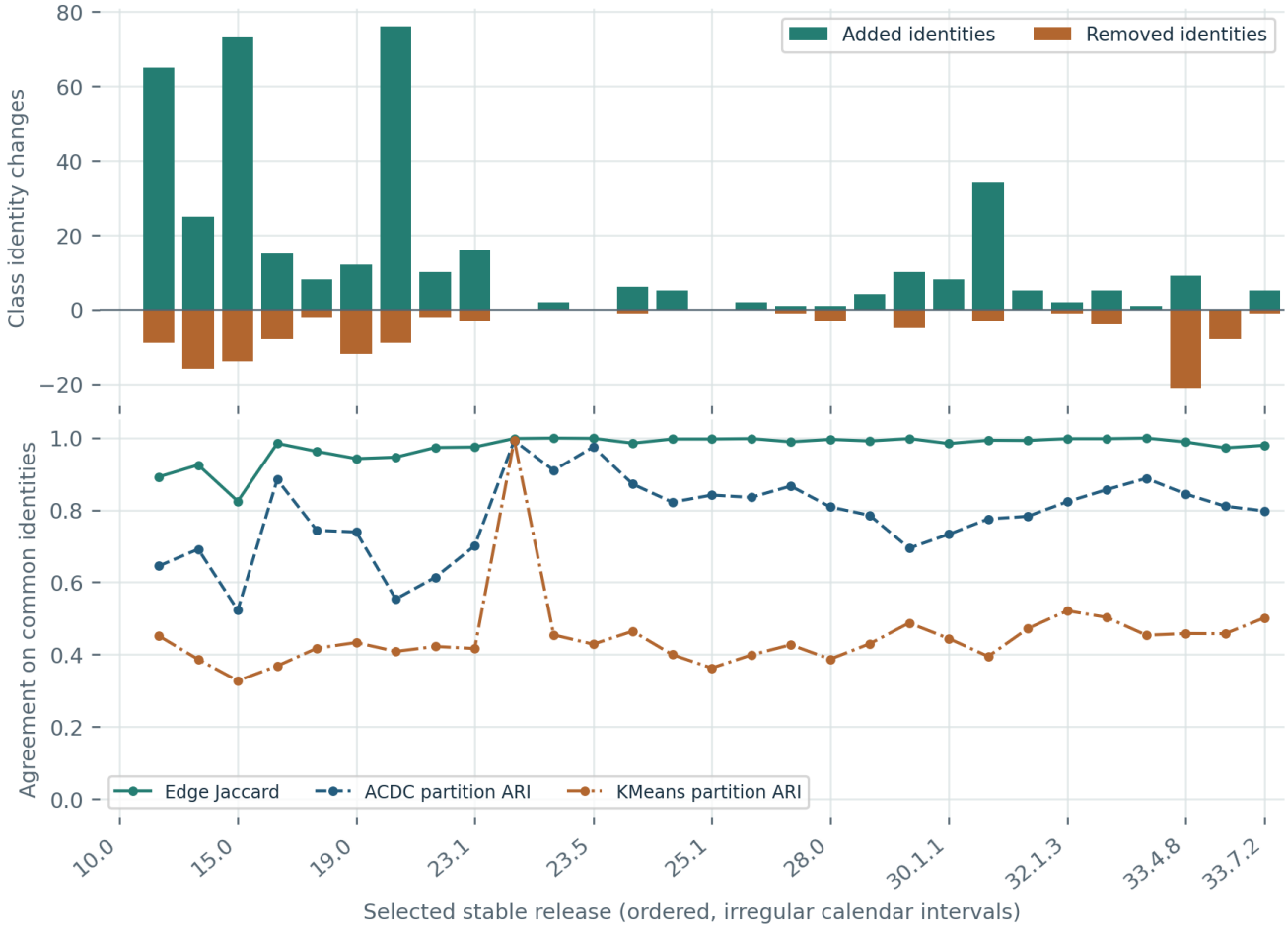


Figure 7. Canonical identity changes and structural agreement between adjacent selected samples. Agreement uses common class identities; dates remain irregular.

The largest sampled identity turnover is 13.0.1 → 15.0: 73 added and 14 removed identities. An identity change can mean introduction, deletion, renaming, package migration or an observable compiler/model change. It is not automatically feature growth or loss. [D10]

The public-suffix implementation remains inside the revised project boundary. Its historical move from com.google.common.net to com.google.thirdparty.publicsuffix no longer excludes that functionality by namespace, although a name-based comparison can still count a rename as one removed and one added identity. [D4; D6]

Common-node edge Jaccard evaluates only dependencies among surviving identities. It can remain high while total edge counts change through newly added or removed classes. Temporal MoJoFM and ARI compare the projected A1/A3 partitions on those same common identities. They are partition stability measures, not evidence of intended-architecture accuracy. [D10; D16]

21 Source evidence for observed transitions

Sample transition	Classes	Edges	Edge change
13.0.1 → 15.0	382 → 441	1639 → 2079	26.85%
19.0 → 21.0	454 → 521	2113 → 2509	18.74%
33.2.1 → 33.4.8	610 → 598	2840 → 2826	-0.49%
33.4.8 → 33.5.0	598 → 590	2826 → 2744	-2.90%

Collections, escaping and I/O between 13.0.1 and 15.0

The audit observes 73 added and 14 removed canonical identities. Added groups include 33 collection types, 9 escaping types and 9 I/O types. Examples include ImmutableRangeMap and ImmutableRangeSet. Source-JAR entry hashes and exact edge differences support the inventory claim. They do not turn every implementation type into a new user feature. [D15]

Graph package observed by the 21.0 sample

Of 76 added identities, 41 are under com.google.common.graph. The common-node edge Jaccard remains 0.9464, so much of the increase accompanies added types while many surviving-node relationships remain stable. The interval also contains an unsampled release; this does not locate the first introduction commit. [D15]

Nullness migration affects the inventory more than primary edges

The source audit finds the earlier internal ElementTypesAreNonnullByDefault declaration and its use in Lists.java, then the later package-level @NullMarked and JSpecify import. There are 15 removed and 4 added annotation types, while primary edges change by only -0.49%. The annotation applications are outside the primary instructor relation model, so the earlier metadata-driven edge spike/fall is not imported into this series. Metadata nodes still affect node and isolate counts. [D15]

Collection support changes at 33.5.0

All 8 removed identities are collection types; seven are nonannotation types and one is GwtTransient. The list includes GWT serialization dependency helpers, ForwardingImmutableCollection and ImmutableEntry. The audit records 87 removed and 5 added edges. These are observed implementation changes, not by themselves proof of architectural repair or public-feature loss. [D15]

22 Reading the latest recovered groups

A partition is more explainable when its members are inspected. The tables below show the five largest clusters from each full-graph recovery at the latest release. Package composition is computed after clustering as a descriptive aid; package labels were not algorithm features. Cluster identifiers are canonical output labels, not authoritative subsystem names. [D7]

ACDC · full

Cluster	Nodes	Dominant packages; example identities
acdc_0011	46	hash: 24; primitives: 7; io: 5 base.Defaults, base.Preconditions, base.Utf8
acdc_0023	46	collect: 41; base: 2; math: 1 base.Predicate, base.Predicates, collect.AbstractListMultimap
acdc_0015	21	util.concurrent: 20; base: 1 base.Function, util.concurrent.AbstractCatchingFuture, util.concurrent.AbstractTransformFuture
acdc_0064	19	graph: 16; collect: 3 collect.HashBiMap, collect.HashMultiset, collect.UnmodifiableIterator
acdc_0123	18	util.concurrent: 18 util.concurrent.AbstractListeningExecutorService, util.concurrent.Callables, util.concurrent.DirectExecutor

KMeans · full

Cluster	Nodes	Dominant packages; example identities
kmeans_0000	85	collect: 19; base: 12; util.concurrent: 12 annotations.Beta, annotations.GwtCompatible, annotations.GwtIncompatible
kmeans_0005	46	io: 9; collect: 8; primitives: 8 base.Defaults, base.PairwiseEquivalence, base.Utf8
kmeans_0014	26	collect: 26 collect.AbstractListMultimap, collect.AbstractMapBasedMultimap, collect.AbstractMultimap
kmeans_0040	23	util.concurrent: 23 util.concurrent.AbstractCatchingFuture, util.concurrent.AbstractExecutionThreadService, util.concurrent.AbstractFuture
kmeans_0023	22	collect: 21; util.concurrent: 1 collect.CollectCollectors, collect.DescendingImmutableSortedMultiset, collect.FluentIterable

For ACDC, the saved original_containers map links normalized labels to the implementation's container names, including any orphan container. Inspect this map before assigning a semantic meaning to a large group. Complete assignments remain available for every condition and release. [D7]

23 Latest-release AI experiment

Recorded model: ChatGPT GPT-6 Astra Pro (fresh delegated session; no seed control exposed). The subject is the same pinned Guava 33.7.2 release used by the classical experiments. A fresh delegated session first wrote a qualitative hypothesis from class names and package counts, then read the other evidence and produced one complete final partition. The initial hypothesis is not a second finalized architecture. [AS pp.3–4; D12]

Exact supplied input	Bytes	SHA-256 prefix
classes.csv	80,871	4351db3d545cd194
dependencies.csv	224,223	2ec951b2d5915223
package_counts.json	629	bfa90ac28edd6ca8
package_dependencies.csv	3,465	6a4670af58a29fde
package_documentation.txt	24,048	612c261362cb659e

The provided graph contains 594 classes and 2,750 directed dependencies. Package documentation is taken from the corresponding source JAR. The protocol withholds ACDC/KMeans partitions, UE decisions and metric outcomes, forbids external browsing, and requires complete one-module-per-class coverage before the coordinator calculates quality. Exact full hashes, paths, activity and outputs are preserved. [D12]

Information provided and reproducibility limits

Names and package documentation are allowed AI evidence; structural algorithms receive graph features only. This information asymmetry prevents a controlled same-input superiority claim.

A fresh session still has general pretrained knowledge. Hidden runtime instructions and model nondeterminism cannot be fully reproduced.

Original revision AI output is preserved in the original Git commit, not reused as this response.

The initial hypothesis and the final explanation are saved separately as initial_hypothesis.md and response.md. The following pages reproduce the actual prompt, enumerate the returned modules, and evaluate that fixed output. Replaying the coordinator's calculations is deterministic; rerunning a live model need not reproduce the same proposal. [D12]

Additional user-visible messages, verbatim

The primary task prompt is reproduced on the next page. The exact additional recorded instruction is reproduced below. It supplies no baseline partition or quality result. The revision has no additional saved completion/status follow-up. [D12]

Launch instruction: Working directory /workspace/scratch/7500a23b5952/software-evolution-ex1. This is a fresh controlled AI architecture-recovery experiment. Your exact task is the full contents of results/ai/prompt.txt. Read that file first and follow it strictly. Read only that prompt and the listed results/ai/inputs evidence; do not inspect any other project data, results, code, baseline architectures, UE decisions or numeric quality scores. First save the initial names-only hypothesis, then inspect the remaining evidence and produce the specified final outputs. Your launch identity is GPT-6 Astra Pro as inherited. The launcher supplies no algorithm outcomes. Complete the recovery now; an independent reviewer evaluates afterward.

SHA-256: d5348c0760560ec08afdd8dcf8b974d39d2938aaa62ea24be97c2e908ae120e5

24 Exact prompt supplied to the AI

The text below is the saved experiment prompt, reproduced without editorial changes. Hidden platform instructions and model internals are not claimed to be part of this record. [D12]

Recover a maintainable conceptual module architecture for the latest selected Guava JRE release, 33.7.2, from the supplied evidence. This is an actual recorded AI architecture-recovery experiment for EPL484 Software Evolution. You are a fresh AI reviewer and must not inspect any ACDC/KMeans partitions, UE decisions, metric outcomes, reports, or other experiment outputs.

Use only the following files under results/ai/inputs (all from the same pinned release): classes.csv (594 canonical outer types), dependencies.csv (2750 binary directed internal class dependencies), package_counts.json, package_dependencies.csv, and package_documentation.txt (unmodified package documentation from the matching source JAR). The primary dependencies are the instructor DependencyExtractor relationships. External libraries have already been excluded and inner/anonymous classes merged. The inventory includes all owned types, including isolated classes and annotation declarations; annotation applications are not edges in this graph. Both com.google.common and Guava-owned com.google.thirdparty.publicsuffix are in scope. A dependency points from a class using a type/member to the class being used. Package declarations and documentation are evidence, not a validated intended architecture. Do not browse external sources or inspect other project files. You may use Python to read and summarize only these inputs and produce the requested outputs.

First inspect classes.csv and package_counts.json only. Record a concise initial architecture hypothesis in results/ai/initial_hypothesis.md before reading dependency/documentation inputs. This hypothesis is qualitative, not a second finalized architecture. Then read the remaining evidence and refine the hypothesis into ONE complete final class partition. Select a practical number of named responsibility modules; do not target a numeric clustering metric. Classes in one package may be split if responsibility boundaries and dependency evidence justify it. Preserve isolated types. Give every supplied class exactly one module and invent no classes. Do not place a class in multiple modules, including utilities.

Write these outputs under results/ai only:
1. assignment.csv with columns class,cluster, containing every supplied class exactly once. Use semantic stable cluster IDs.
2. architecture.json with schema_version, subject.release, modules (id, name, responsibility, exact class_count, boundary_rationale), cross_module_observations, changes_from_initial_hypothesis, and limitations. Cite concrete class/package relationships and graph counts where relevant, separating interpretation from observation.
3. response.md: complete final explanation, including how additional evidence changed or confirmed the names-only hypothesis; package-structure bias; confidence limits; and which intended-design claims need maintainer validation. Do not claim superiority or architectural accuracy without ground truth. Explain how the proposal helps a human understand Guava.
4. activity.json: exact input paths read, tool/method summary, model identity available from your runtime (do not invent unavailable identifiers), and confirmation that baseline partitions and numeric quality scores were not consulted.

Validate partition coverage against classes.csv. Do not compute MQ/cohesion/coupling or compare with baseline algorithms; the coordinator evaluates afterward on matching graph scopes. Leave unrelated files untouched. No subdelegation is needed for this small controlled experiment.

`Prompt SHA-256: f9cb3f65ff158253eb15acb2580e8f2b4adbd7fd337307b1a84c6c79c218810a`

25 The AI's proposed conceptual modules

The finalized AI assignment contains **18 modules covering all 594 supplied classes exactly once**. Responsibilities below are the model's interpretation of the supplied names, documentation and dependencies. They are not maintainer-approved design constraints. The machine-readable architecture also records each boundary rationale and its supporting observations. [D12]

Module	Classes	Responsibility excerpt
Core values and programming contracts	57	Basic values, argument checks, functional adapters, text helpers, timing, reference lifecycle support, and shared API/platform annotation declarations.
Core collections and ordering	127	Ordinary maps, bimaps, lists, sets and queues; immutable implementations of those types; collection views, iteration, ordering, forwarding decorators,…
Multimaps and multisets	59	Multiple-value mappings and occurrence-counting collections, including their interfaces, mutable and immutable implementations, filtering views, sorted…
Two-dimensional tables	16	Row/column-keyed Table APIs, mutable and immutable representations, forwarding views, and table-specific collectors.
Ranges and interval collections	15	Range endpoints and bounds, discrete domains, contiguous sets, and range-set/range-map representations.
Caching and removal policy	22	Cache APIs, loading, construction and configuration, eviction/removal policy, statistics, and local cache implementation support.
Futures and execution runtime	58	Future values and composition, listening executors, task submission and sequencing, executor adapters, thread factories, execution exceptions,…
Service lifecycle management	8	Service state/lifecycle APIs, service implementation templates, service groups, and lifecycle listener dispatch.
Synchronization and execution limits	15	Atomic state utilities, locks and monitors, striped/cycle-detecting synchronization, rate limiting, and time limits.
Graph models and traversal	53	Graph, network and value-graph APIs, builders, mutable/immutable representations, adjacency/connection machinery, and traversal.
Hash functions and probabilistic membership	33	Hash functions and codes, hashers, primitive sinks/funnels, digest and checksum adapters, hashing streams, and Bloom filters.
Streams, files and resource I/O	37	Byte/character sources and sinks, streams and readers, file/resource operations, resource closing, line processing, and base encoding.
Primitive value utilities	25	Primitive-type utilities and conversions, unsigned value types, immutable primitive arrays, and associated platform helpers.
Arithmetic and statistics	16	Arithmetic and rounding over numeric values, quantiles, statistical summaries/accumulators, and linear transformations.
Network identifiers and protocol values	10	IP addresses, host/port specifications, domain names and owned public-suffix support, media types, and HTTP constants.
Text escaping policies	16	Generic character/Unicode escape abstractions, escaper builders, and HTML, XML, and URL escaping policies.
Reflection and generic types	16	Generic type capture/resolution, type tokens, invocation and parameter inspection, type-to-instance maps, class-path inspection, and reflection helpers.
Event subscription and dispatch	11	Event bus APIs, synchronous/asynchronous dispatch support, subscription declarations, subscriber discovery, and exception handling.

Complete class-level output: results/ai/assignment.csv. Complete semantic response: results/ai/response.md. Initial hypothesis: results/ai/initial_hypothesis.md. Machine-readable module definitions, refinements and limitations: results/ai/architecture.json. [D12]

26 AI quality and comparison

Partition	Graph	k	Cohesion	Coupling	MQ	Inter-edge %
AI final	Full	18	0.0966	0.0023	0.0943	40.7%
A1	Full	135	0.2575	0.0022	0.2553	63.9%
A3	Full	43	0.1852	0.0062	0.1790	65.9%
AI projected	Retained	18	0.0907	0.0012	0.0895	26.2%
A2	Retained	149	0.2431	0.0014	0.2417	53.1%
A4	Retained	43	0.2052	0.0031	0.2021	53.1%

The AI's full assignment is evaluated on the same graph as A1/A3. For A2/A4 comparisons, the same AI assignment is projected onto retained nodes and evaluated on retained edges, without asking the model for another architecture. Cross-scope scores are not compared as though they measured the same graph. Information supplied to the methods also differs: the AI sees names and documentation, while KMeans receives structural features. [D12]

Baseline	Graph	AI → baseline MoJoFM	Baseline → AI MoJoFM	ARI
A1	full	28.57	61.63	0.1666
A2	retained	32.85	63.18	0.2008
A3	full	32.52	61.63	0.1701
A4	retained	33.82	63.36	0.1535

Three concrete refinements from the added evidence

Four responsibilities: collections_core, multimaps_multisets, tables, and ranges. Documentation names multiple collection contracts and range semantics; direct references between Multimap and Multiset support their joint family, while table-specific implementations and collectors support a separate Table responsibility. Shared helpers and immutable representations prevent interpreting the split as clean compilation layers.

async_execution, service_lifecycle, and synchronization_limits; two blocking forwarding adapters join collections_core. Futures and executor composition share direct implementation relationships; lifecycle templates and ListenerCallQueue form a recognizable responsibility; atomic/lock/rate/time APIs express coordination. The two forwarding adapters directly reuse core collection forwarding types.

Separate primitive_values and mathematics. Documentation distinguishes representation/storage from arithmetic, and all five observed math/primitives cross-package edges point toward primitive utilities.

Interpretation and confidence

AI MQ is 0.0943 on the full graph and 0.0895 after projection. The AI uses 18 modules and A3 uses 43 clusters; their adjusted Rand agreement is 0.1701. Aggregate MQ does not specify which classes are grouped together. Granularity and information differ: the AI sees names and source documentation, while the structural algorithms follow their own graph objectives. No acyclic build decomposition or intended-design accuracy is established. [D9; D12]

Direction matters: AI → A1 MoJoFM is 28.57 but A1 → AI is 61.63. Moves can be necessary to subdivide coarse groups, whereas joins can merge fine groups; the target also determines normalization. Reporting one favorable direction would hide this asymmetry. [D12; D16]

Partition agreement and coordinator evaluation details: results/ai/comparisons.csv; metrics.json; retained_metrics.json. The live model's complete limitations and cross-module observations remain in architecture.json and response.md. [D12]

27 Performance and scaling evidence

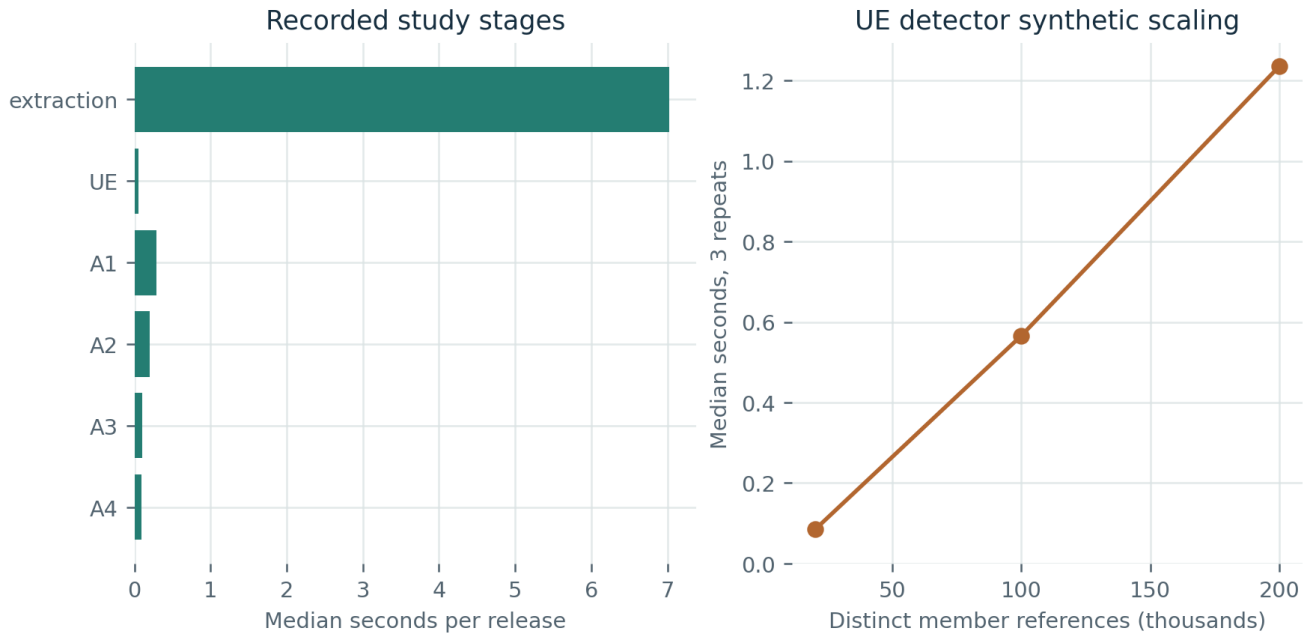


Figure 8. Actual study-stage timings and a separate controlled synthetic UE benchmark. They measure different scopes and are not speedup ratios.

Study stage	Median / release (s)	Maximum (s)	30-release sum (s)
extraction	7.0253	7.8311	197.228
UE	0.0555	0.1078	1.804
A1	0.2890	0.3241	8.501
A2	0.1951	0.2541	5.875
A3	0.1023	0.1135	2.925
A4	0.0943	0.1167	2.750

Synthetic N	Member references	Median (s)	Process high-water MiB
1,000	20,000	0.0847	34.0
5,000	100,000	0.5662	117.6
10,000	200,000	1.2353	221.8

Study extraction includes JVM startup and I/O; UE study time includes member-CSV parsing; clustering is wall time around each adapter. Downloads, the pilot, report generation and tests are not included in the stage sums. Synthetic timings isolate the detector including validation and diagnostics, use three repeats and fixed degree-20 indexed graphs, and verify expected scores for every generated node. [D11]

Memory is cumulative process high-water RSS, including constructed inputs and prior result objects; it is not per-call peak allocation. Times depend on this environment and do not prove a speedup over the old executable, which was not used as a benchmark or correctness oracle. The measurements support practicality within the tested input sizes. [D11]

28 Course-grounded interpretation

Implemented structure, intended design and drift

This is architecture reconstruction by reverse engineering: a lower-level static dependency graph is converted into a higher-level decomposition. The architecture-evolution lecture distinguishes implemented, intended and described architecture, as well as static and dynamic views. Our partitions are candidate views of the implementation. Demonstrating erosion would require an intended design constraint and evidence that changes violate it. MQ trends alone do not provide that evidence. [L5 PDF pp.9,12,14,18–19]

Lehman's laws are a lens, not an automatic conclusion

Changes in class counts and dependencies are consistent with continuing change and may motivate inspection of structural complexity. They cannot by themselves verify continuing functional growth, declining user-perceived quality, invariant effort or organizational stability. Those claims need requirements, usage, effort and process evidence. The lectures also restrict the empirical laws to E-type systems; a Java utility library should not be classified as E-type without explaining its real-world adaptation context. [L12 pp.33–48]

Operational definitions and traceability

The requirements lectures emphasize ambiguity, measurable statements and traceability. “Internal,” “class,” “dependency,” “stable release,” “omnipresent” and “quality” therefore have explicit operational definitions here. The same term can mean something different in requirements relationships, runtime interactions and class-file structure. Stored inputs, parameters and per-cluster terms make a claim auditable, but traceability alone does not guarantee its correctness. [L3 pp.9–19,27–33; L4 pp.11–15]

Material limits of this experiment

The 30 observations form a systematic sparse sample, not a random sample or a complete history. The artifact and namespace boundary omits external interactions and other modules. Static analysis cannot observe every reflective or dynamically created dependency. Published bytecode can reflect different compiler versions and inlining. The UE replacement depends on resolved member usage and its fixed thresholds; it is not interchangeable with the older Noise method. ACDC granularity and KMeans' geometry/pilot extrapolation affect the recovered structure. [D2; D4; D6; D7]

The latest extraction records 659 unresolved member-use events (494 distinct recorded references); 335 events cross normalized class boundaries. These diagnostics quantify a known evidence gap. Their ratio to raw observed calls includes external and intraclass events, so it is not a completeness or false-negative rate. [D14]

Automated metrics remove repeated manual calculation, but they do not remove modeling judgment. Without an expert architecture, partition agreement is not accuracy. The AI output adds a semantic interpretation and its own prompt/model constraints. The defensible conclusions are the observed structural results under these declared choices. [AS pp.3–4; L3 pp.16–19]

29 Reproduce and audit the study

Use the repository's pinned Python dependencies and Java toolchain instructions first. The following sequence reconstructs the data-to-report chain; --offline verifies an existing artifact cache. Exact flags and prerequisites are documented in the repository README and acquisition guide.

```
python scripts/unpack_data.py
python scripts/select_versions.py --check
python scripts/fetch_data.py --offline
python scripts/fetch_tools.py
python scripts/build_java.py
python scripts/run_study.py --phase all
python scripts/audit_change_points.py
python scripts/audit_revision.py
python scripts/evaluate_ai.py
python scripts/validate_results.py
python scripts/build_report.py
```

Artifact path	What to verify
config/versions.json	30 distinct families with immutable representatives, URLs, hashes and catalog decisions.
data/graphs/<version>/	Original instructor input/raw output; normalization; typed and binary edges; aliases; separate supplemental member evidence and comparisons.
results/versions/<version>/	UE scores, retained graph, exactly one A1–A4 assignment and its execution/quality details.
results/*.csv	30 size rows, 120 metric rows, 60 same-graph rows, 29 transitions, 180 stage timings, 178 directional comparison pairs.
results/run_manifest.json	Input/config hashes, algorithm-source hashes, Python, Java and package versions.
results/ai/	Actual AI prompts, provided input, output, coverage audit and comparison.
report/report_manifest.json	Hashes of files read by the report and generated PDF/figures; no recomputed partitions.

Recorded software: Python 3.12.14; matplotlib 3.10.8; numpy 2.3.5; reportlab 4.4.9; scikit-learn 1.8.0; scipy 1.17.0. The full Java toolchain identity and operating-system string are retained in run_manifest.json. Stable inputs, ordering and seeds make the procedure repeatable; environment changes can still affect floating-point details and wall time. [D11]

`Selection SHA-256: 733cfefe8582f23ee0dba1349773ceeaf67e746cf8892f7e21fe014cb3e62410`
`Configuration SHA-256: c2d715ba45c0ee9445e8d07bfba238c3a38d1dcf656aab909f828a7617ccb35f`

The published repository packages raw artifacts and full per-release graph/result evidence in deterministic ZIP bundles under archives/versions. Unpacking restores the documented paths; summaries, source code, figures and guides remain directly readable. No source artifact is replaced by a derived summary.

Independent audit status: **passed**, 8,348,851 evidence checks and 0 failures. A separate standard-library implementation recalculates exact-rational UE, lecture MQ and contingency-table ARI and directional MoJoFM from stored evidence. It checks all 120 final partitions, all 14 pilot inertias and the fixed AI output. This establishes internal consistency, not completeness of static analysis or intended-design accuracy. [D13]

Appendix A1 Complete quality results

Selected releases 1–10 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
10.0	A1	317	55	0.2777	0.0044	0.2733	57%	35
10.0	A2	311	65	0.2173	0.0029	0.2144	52%	20
10.0	A3	317	23	0.1741	0.0067	0.1675	57%	45
10.0	A4	311	23	0.1981	0.0075	0.1905	52%	63
12.0.1	A1	373	70	0.2783	0.0055	0.2728	61%	29
12.0.1	A2	365	79	0.2523	0.0033	0.2490	53%	24
12.0.1	A3	373	27	0.1650	0.0074	0.1576	63%	36
12.0.1	A4	365	27	0.1828	0.0037	0.1791	50%	59
13.0.1	A1	382	80	0.2739	0.0047	0.2692	64%	36
13.0.1	A2	374	89	0.2397	0.0030	0.2367	58%	29
13.0.1	A3	382	28	0.1707	0.0083	0.1624	69%	53
13.0.1	A4	374	28	0.1907	0.0060	0.1847	56%	72
15.0	A1	441	93	0.3111	0.0041	0.3070	64%	41
15.0	A2	430	100	0.2976	0.0027	0.2949	55%	29
15.0	A3	441	32	0.1784	0.0065	0.1720	67%	51
15.0	A4	430	32	0.1647	0.0048	0.1598	54%	72
16.0.1	A1	448	95	0.3141	0.0038	0.3104	64%	43
16.0.1	A2	438	101	0.2911	0.0026	0.2885	55%	30
16.0.1	A3	448	33	0.1513	0.0075	0.1437	70%	51
16.0.1	A4	438	33	0.1609	0.0042	0.1568	53%	62
18.0	A1	454	99	0.3116	0.0036	0.3080	63%	43
18.0	A2	444	106	0.3042	0.0024	0.3019	55%	40
18.0	A3	454	33	0.1573	0.0070	0.1502	71%	56
18.0	A4	444	33	0.1629	0.0056	0.1573	59%	84
19.0	A1	454	97	0.3340	0.0037	0.3303	63%	54
19.0	A2	441	105	0.2938	0.0023	0.2915	54%	31
19.0	A3	454	33	0.1954	0.0057	0.1897	69%	51
19.0	A4	441	33	0.1953	0.0041	0.1912	53%	80
21.0	A1	521	102	0.3185	0.0033	0.3151	64%	50
21.0	A2	505	113	0.2579	0.0019	0.2561	52%	39
21.0	A3	521	38	0.1735	0.0071	0.1664	71%	57
21.0	A4	505	38	0.1873	0.0041	0.1831	55%	92
22.0	A1	529	102	0.3158	0.0032	0.3126	64%	44
22.0	A2	513	115	0.2588	0.0018	0.2570	52%	34
22.0	A3	529	39	0.1828	0.0056	0.1773	69%	60
22.0	A4	513	39	0.1997	0.0031	0.1966	50%	88
23.1	A1	542	106	0.3151	0.0034	0.3117	63%	56
23.1	A2	525	117	0.2751	0.0018	0.2733	54%	44
23.1	A3	542	40	0.1600	0.0049	0.1551	70%	56
23.1	A4	525	40	0.1862	0.0037	0.1824	55%	98

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

Appendix A2 Complete quality results

Selected releases 11–20 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
23.2	A1	542	106	0.3154	0.0034	0.3120	63%	57
23.2	A2	525	117	0.2752	0.0019	0.2733	54%	44
23.2	A3	542	40	0.1599	0.0049	0.1550	70%	56
23.2	A4	525	40	0.1825	0.0034	0.1791	54%	80
23.4	A1	544	107	0.3189	0.0034	0.3156	64%	52
23.4	A2	528	116	0.2825	0.0018	0.2807	54%	33
23.4	A3	544	40	0.1457	0.0060	0.1397	72%	71
23.4	A4	528	40	0.1766	0.0035	0.1731	55%	92
23.5	A1	544	107	0.3169	0.0033	0.3136	64%	49
23.5	A2	528	116	0.2828	0.0018	0.2809	54%	33
23.5	A3	544	40	0.1660	0.0057	0.1603	71%	71
23.5	A4	528	40	0.1744	0.0040	0.1704	55%	95
24.0	A1	549	110	0.3169	0.0037	0.3132	64%	55
24.0	A2	532	119	0.2894	0.0019	0.2875	54%	46
24.0	A3	549	40	0.1715	0.0067	0.1648	71%	63
24.0	A4	532	40	0.1769	0.0037	0.1732	56%	94
24.1	A1	554	109	0.3202	0.0034	0.3168	64%	54
24.1	A2	537	119	0.2733	0.0019	0.2714	55%	35
24.1	A3	554	40	0.1650	0.0059	0.1591	70%	71
24.1	A4	537	40	0.1923	0.0037	0.1886	57%	93
25.1	A1	554	110	0.3228	0.0035	0.3193	65%	49
25.1	A2	537	120	0.2776	0.0020	0.2756	55%	35
25.1	A3	554	40	0.1655	0.0052	0.1603	67%	58
25.1	A4	537	40	0.1685	0.0043	0.1643	56%	78
26.0	A1	556	110	0.3125	0.0036	0.3089	65%	48
26.0	A2	539	120	0.2852	0.0021	0.2831	55%	43
26.0	A3	556	41	0.1520	0.0072	0.1448	70%	71
26.0	A4	539	41	0.1963	0.0035	0.1928	55%	101
27.1	A1	556	110	0.3260	0.0034	0.3226	64%	55
27.1	A2	538	120	0.2849	0.0020	0.2829	54%	47
27.1	A3	556	41	0.1754	0.0066	0.1688	71%	62
27.1	A4	538	41	0.1918	0.0036	0.1883	51%	106
28.0	A1	554	111	0.3127	0.0035	0.3092	65%	49
28.0	A2	535	121	0.2822	0.0021	0.2801	55%	34
28.0	A3	554	40	0.1573	0.0057	0.1516	69%	54
28.0	A4	535	40	0.1809	0.0043	0.1766	58%	96
28.2	A1	558	112	0.3297	0.0035	0.3262	65%	48
28.2	A2	539	121	0.2802	0.0019	0.2782	55%	34
28.2	A3	558	41	0.1532	0.0071	0.1461	73%	60
28.2	A4	539	41	0.1848	0.0040	0.1808	57%	89

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

Appendix A3 Complete quality results

Selected releases 21–30 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
29.0	A1	563	117	0.3247	0.0035	0.3212	65%	49
29.0	A2	543	126	0.2965	0.0018	0.2947	56%	32
29.0	A3	563	41	0.1704	0.0054	0.1650	68%	74
29.0	A4	543	41	0.1669	0.0034	0.1635	57%	99
30.1.1	A1	571	120	0.3266	0.0036	0.3229	66%	43
30.1.1	A2	551	127	0.3029	0.0023	0.3006	57%	27
30.1.1	A3	571	42	0.1766	0.0065	0.1701	72%	67
30.1.1	A4	551	42	0.1916	0.0038	0.1879	57%	82
31.0.1	A1	602	149	0.2704	0.0022	0.2683	65%	45
31.0.1	A2	581	156	0.2376	0.0014	0.2362	57%	27
31.0.1	A3	602	44	0.1759	0.0058	0.1701	70%	91
31.0.1	A4	581	44	0.1784	0.0037	0.1747	60%	119
32.0.1	A1	607	149	0.2541	0.0021	0.2520	64%	49
32.0.1	A2	585	157	0.2348	0.0013	0.2335	55%	31
32.0.1	A3	607	44	0.1632	0.0069	0.1562	71%	100
32.0.1	A4	585	44	0.1987	0.0037	0.1951	58%	138
32.1.3	A1	608	151	0.2580	0.0020	0.2560	65%	47
32.1.3	A2	586	159	0.2332	0.0014	0.2318	55%	30
32.1.3	A3	608	44	0.1538	0.0051	0.1487	69%	96
32.1.3	A4	586	44	0.1832	0.0038	0.1794	57%	127
33.1.0	A1	609	149	0.2407	0.0019	0.2387	65%	46
33.1.0	A2	586	160	0.2234	0.0014	0.2220	56%	32
33.1.0	A3	609	44	0.1784	0.0071	0.1712	70%	109
33.1.0	A4	586	44	0.1798	0.0038	0.1760	59%	134
33.2.1	A1	610	149	0.2398	0.0018	0.2379	65%	46
33.2.1	A2	587	160	0.2328	0.0013	0.2315	54%	37
33.2.1	A3	610	45	0.1655	0.0060	0.1595	74%	102
33.2.1	A4	587	45	0.1678	0.0041	0.1637	58%	120
33.4.8	A1	598	138	0.2662	0.0021	0.2641	65%	47
33.4.8	A2	575	151	0.2392	0.0011	0.2381	52%	42
33.4.8	A3	598	44	0.1709	0.0047	0.1662	69%	93
33.4.8	A4	575	44	0.1795	0.0040	0.1755	57%	120
33.5.0	A1	590	134	0.2687	0.0023	0.2664	63%	53
33.5.0	A2	569	149	0.2379	0.0014	0.2365	54%	32
33.5.0	A3	590	43	0.1871	0.0060	0.1811	70%	95
33.5.0	A4	569	43	0.1911	0.0027	0.1884	51%	116
33.7.2	A1	594	135	0.2575	0.0022	0.2553	64%	46
33.7.2	A2	572	149	0.2431	0.0014	0.2417	53%	35
33.7.2	A3	594	43	0.1852	0.0062	0.1790	66%	85
33.7.2	A4	572	43	0.2052	0.0031	0.2021	53%	119

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

References and source index

Supplied teaching material

[AS] EPL484, Fall 2026, Project 1 instructions, 5 PDF pages. Eligibility p.1; data pp.2–3; AI and report pp.3–5.

[TUT] Project 1 tutorial, 12 PDF pages. Deadline p.2; relation types p.6; ACDC pp.7–8; clustering pp.10–12.

[L12] Lectures 1–2: Introduction, 50 PDF pages. Evolution, maintenance and S/P/E classification pp.15–33; Lehman's laws and limits pp.35–48.

[L3] Lecture 3: Requirements, Part 1, 38 PDF pages. Measurable definitions/automation pp.9–19; relationship and traceability models pp.27–33.

[L4] Lecture 4: Requirements, Part 2, 33 PDF pages. Traceability and its limits pp.11–15; feedback evidence pp.22–31.

[L5] Lecture 5: Architecture Evolution, 34 PDF pages. PDF p.9 = printed slide10; after the missing slide6, PDF and printed numbers differ by one.

[L67] Lectures 6–7: Architecture Reconstruction, 50 PDF pages. MQ equations and worked example pp.15–17; ACDC pp.19–24; older Noise method pp.46–47.

Research and primary documentation

[UE] Yano and Matsuo (2020), Moderate Detection and Removal of Omnipresent Modules in Software Clustering, ICSME, pp.662–666. Equations1–3: p.663; procedure: p.664. https://doi.org/10.1109/ICSME46990.2020.00068

[NOISE] Constantinou, Kakarontzas and Stamelos (2015), JSS107, pp.142–157. Older significance-based method, distinguished from this UE replacement. https://doi.org/10.1016/j.jss.2015.05.065

[ACDC] Original ACDC author archive, preserved and SHA-256 pinned in third_party/acdc.
https://www.eecs.yorku.ca/~bil/downloads/acdc.tar

[SK] scikit-learn 1.8 KMeans API and normalize API; actual installed version is recorded by the run.
https://scikit-learn.org/1.8/modules/generated/sklearn.cluster.KMeans.html

[JVMS] Java Virtual Machine Specification21, §5.4.3.2–5.4.3.4: field and method resolution.
https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-5.html

[G1] Guava source repository and project overview. https://github.com/google/guava

[G2] Guava ReleaseHistory, with linked historical release notes. https://github.com/google/guava/wiki/ReleaseHistory

Repository evidence cited in this report

[D1] results/sizes.csv; metrics.csv; versions/*/*_metrics.json

[D2] docs/omnipresent.md; src/evolution/omnipresent.py; results/versions/*/ue.json

[D3] results/same_graph_comparisons.csv; versions/*/*_projected_to_retained_metrics.json

[D4] docs/project_selection.md; config/versions.json; data/provenance/tag_inventory.json; project_eligibility.json

[D5] docs/data_acquisition.md; scripts/fetch_data.py; data/provenance/acquisition.json and source/Git audits

[D6] docs/modeling_decisions.md; extractor source; data/graphs/*/provenance.json and diagnostics

[D7] docs/course_traceability.md; src/evolution/clustering.py; config/analysis.json; A1–A4 run metadata

[D8] results/pilot/pilot.json and associated pilot assignments

[D9] src/evolution/metrics.py; tests; results/versions/*/*_quality/

[D10] results/evolution.csv; canonical class and edge manifests

[D11] results/runtimes.csv; run_manifest.json; performance/ue_benchmark.json

[D12] results/ai/experiment.json and its exact prompt/input/output/quality artifacts

[D13] scripts/validate_results.py; results/validation.json, independent artifact audit

[D14] docs/ue_case_study.md; results/performance/case_checks.json, exact-fraction source-backed UE cases

[D15] scripts/audit_change_points.py; results/change_points.json, source and graph-difference audit

[D16] docs/mojofm.md; src/evolution/mojofm.py; saved directional partition agreement tables

[D17] results/baseline_v1; original full study at Git commit d970305; revised config and primary extractor comparison